

Git Push Quick Guide — How to push changes t

Practical, step-by-step commands + how to know what to run

Example repo: <https://github.com/learntospeak/langylearner> (replace with your repo if different)

One-time setup for a new repo (after installing Git)

1) Open Terminal/PowerShell in your project folder.

2) Initialize and make the first commit:

```
git init
```

```
git add -A
```

```
git commit -m "feat: first public lesson build"
```

3) Add your GitHub remote (HTTPS example):

```
git remote add origin https://github.com/<USERNAME>/<REPO>.git
```

4) Set the main branch name and push it:

```
git branch -M main
```

```
git push -u origin main
```

Everyday push flow (after you've already connected the repo)

```
# See what changed (red/green files)
git status
```

```
# Make sure you're up to date before pushing
git pull --rebase origin main
```

```
# Stage and commit everything you changed
git add -A
git commit -m "chore: updates"
```

```
# Send it to GitHub
git push
```

How to know which command to use

- First push ever from this machine? Run: `git remote -v`
 - If it prints nothing: add the remote (`git remote add origin ...`).
 - If it prints an SSH/HTTPS URL: remote is set; no need to add again.
- Unsure which branch you're on? Run: `git branch --show-current`
 - If not 'main', replace 'main' in commands with your current branch.
- "nothing to commit, working tree clean"
 - You have no file changes. Save a file or change something, then repeat:
`git add -A && git commit -m "msg" && git push`
- "Your branch is ahead of 'origin/main' by N commit(s)"
 - That means you need to push: `git push`
- "non-fast-forward" or "rejected"
 - Pull remote commits cleanly, then push:
`git pull --rebase origin main`
resolve any conflicts, then
`git push`
- Files renamed but GitHub didn't update the case (Windows / macOS)
 - Tell Git to respect case-only renames:
`git config core.ignorecase false`
stage the rename and commit again

Verify your push

On GitHub: open your repo → “Code” tab → check the latest commit time.
Locally: `git log --oneline -n 3`

For GitHub Pages, force-refresh the site with a cache-buster:
`https://<username>.github.io/<repo>/lesson.html?v=12345`

Common issues & quick fixes

- Authentication / 403
 - If using HTTPS, GitHub may ask for a Personal Access Token as your password.
- “Nothing changed on the live site”
 - Confirm the commit is on the correct branch (main).
 - GitHub Pages may take a minute; hard refresh or add ?v= timestamp.
 - Ensure paths are relative (fetch('stories.json'), not '/stories.json').
- Service worker caching old files
 - Bump CACHE_VERSION in your service worker, then push and hard refresh.
- Large files
 - Avoid committing node_modules, build artifacts, or big media.
 - Use .gitignore and/or Git LFS for large binary assets.

Ultra-short checklist (copy/paste)

```
git status
git pull --rebase origin main
git add -A
git commit -m "update: <what you changed>"
git push
```